



Birzeit University

Faculty of Engineering and Technology

Department of Electrical and Computer Engineering

ENCS4320 — Applied Cryptography (Term 1253)

SecureVault: A Multi-User Encrypted Document Exchange System

1. Overview

Your team will build a system that lets people store private documents on a server they do not trust, and share those documents with each other. The server will hold every byte your users upload — and must never be able to read a single one of them.

This document **specifies the problem, not the solution**: it states what your system must do and what it must guarantee, and it deliberately does not prescribe algorithms, message layouts, or program structure. Those are engineering decisions, and they belong to your team.

For each decision, Section 7 sets out the realistic options and the questions that separate them. Your team chooses, records the reasoning, and defends it during the presentation. A well-argued design that is slightly slower will be marked above an excellent design whose authors cannot explain it.

Two teams following this specification carefully may hand in very different systems. That is expected, and it is the intended outcome.

2. The System, in One Scenario

Before the requirements, here is what your finished system does. If you can make this story happen safely, you have built the project.

Layla opens your client and creates an account with a username and a password. Nothing that leaves her machine would let anyone recover that password, now or after the server is breached.

Omar does the same on his own machine. The two have never met online before.

Layla logs in and uploads *thesis-draft.pdf*. The server stores it and can tell you its size and when it arrived — but not one word of its contents.

Layla shares the document with Omar. She names him; the system does the rest. The server carries the traffic and learns nothing new.

Omar logs in, sees the shared document, downloads it, and his client tells him two things: the file is intact, and it came from Layla. He can show a third party that it came from Layla — she cannot later deny sending it.

Meanwhile, an attacker who controls the network flips a single byte of the ciphertext. Omar's client refuses the file and says so. The attacker changes the recorded file name instead; the client refuses again. The attacker replays yesterday's upload; it is rejected as stale. The attacker steals the server's entire database and starts guessing passwords; the arithmetic in your report explains why they will still be guessing long after anyone cares.

Everything else in this document exists to make that scenario true and to make you argue for how you achieved it.

3. Learning Objectives

On completing this project, you will be able to:

1. Turn a set of security requirements into a concrete cryptographic design, and justify every choice against the alternatives you rejected.
2. Implement cryptographic algorithms correctly and demonstrate that they behave as their designers intended.
3. Store user credentials so that a full database breach does not yield passwords at scale, and quantify that claim.
4. State a threat model precisely, and reason about which attacks your design stops and which it does not.
5. Defend a design under questioning, and adapt it in response to a new requirement.

4. Participants and Threat Model

Three parties appear in this system, and stating their capabilities precisely is the foundation of everything that follows.

Party	Capabilities and Assumptions
Users	Users run your client software on machines assumed to be trustworthy. Each user has a password known only to them, and an attacker cannot read an honest user's memory or disk.
The server	The server stores accounts and documents and relays them between users. It is honest-but-curious: it follows your protocol faithfully and neither deletes nor fabricates data, yet it reads everything it is given, and its entire database may fall into an attacker's hands.
The attacker	The attacker observes and modifies all network traffic, replays old messages, registers accounts of their own, and may obtain a complete copy of the server's stored data.

Out of scope: malware on a user's own machine, coercion of a user, traffic-analysis of message sizes and timing, and denial of service. You may mention these in your limitations section, but you are not required to defend against them.

5. What the System Must Do

Five operations. For each, the requirement is stated; how you meet it is the subject of Section 7.

5.1 Sign-Up

- A new user registers with an identity and a password of their choosing.
- The server can check a password at login, yet stores nothing from which passwords can be recovered in bulk.
- Two users who pick the same password must not look identical in the stored data.
- An attacker holding the whole credential store must not be able to test candidate passwords quickly.

5.2 Login

- A returning user proves possession of their password and obtains whatever session state your design needs.
- A failed login reveals nothing about whether the account exists — not through its message, and not through how long it takes.

5.3 Key Management

- Each user holds cryptographic material that identifies them to other users.
- A user can obtain another user's public material and be confident it truly belongs to that person. Remember that the server handing it over is not trusted to be honest about it.
- Private material must be protected at rest. Decide where it lives and what recovers it when the user has only their password.

5.4 Upload

- A user uploads a document; the server does not learn its contents.
- Each document carries descriptive information — file name, type, size, timestamp, owner. Decide what the server legitimately needs in the clear, and make certain that none of it can be altered undetected.

5.5 Share, Retrieve and Verify

- A user grants a named user access to a stored document in the server. The recipient can read it.
- The recipient learns who produced the document, and can demonstrate that to a third party.
- Any modification — to the ciphertext, the metadata, or the claimed sender — is **detected and rejected**, never silently accepted.
- A document captured earlier and re-delivered later is rejected as stale.

6. What the System Must Guarantee

Your report must state, for each property, the specific mechanism in your design that provides it.

Property	Meaning in this system
Confidentiality	Nobody but the intended users learns document contents — the server included.
Integrity	Any alteration is detected before the contents are used.
Data-origin authentication	The recipient learns which user produced the document.
Non-repudiation	The recipient can prove authorship to a third party. Think hard about which mechanisms can deliver this and which cannot.
Credential protection	A stolen credential database does not yield passwords at scale.
Freshness	Replayed or reordered objects are rejected.
Metadata binding	Anything the server sees in the clear cannot be altered, or moved to a different document, undetected.

These properties interact. A design that satisfies each one in isolation can still fail when they are combined — the classic example being a scheme that encrypts correctly and authenticates correctly but does the two in the wrong order. Explaining how yours fit together is part of the assessment.

7. The Decisions You Must Make

This section is the heart of the project. Each subsection names the realistic options and the questions that distinguish them. Choose one, and be ready to explain why you did not choose the others.

7.1 Protecting Passwords

Options: PBKDF2, bcrypt, scrypt, Argon2, or a construction of your own built on a hash you implement.

- What separates these is whether they are memory-hard, how the work factor is expressed, and how well each resists an attacker with a GPU. Which of those matter for your threat model?
- What work factor did you set, and what measurement justifies it? A number with no measurement behind it is not a justification.
- How is the per-user random value generated, sized and stored — and why is it not secret?
- Does anything derived from the password serve a second purpose in your design? If so, what breaks when the user changes their password?

7.2 Protecting Document Contents

Options: a dedicated authenticated-encryption mode, or a cipher and a MAC that you compose yourself. Among modes you might weigh GCM, CCM, or a stream cipher paired with a one-time MAC; among block ciphers, AES at a key size you justify.

- If you compose encryption and authentication yourself, in which order — and why is that order the safe one?
- How many distinct keys does one document need, and where does each come from?
- How are nonces/IVs produced, and what guarantees they never repeat under one key — **including after a crash or a restart**? This is the single most common way real systems of this kind fail.

7.3 Public-Key Cryptography

Your system must make use of **elliptic curve cryptography** in at least one of these roles. Options: key agreement by Diffie–Hellman over an elliptic curve or over a finite field; key transport based on RSA; and, for signatures, ECDSA, RSA, or DSA. You need not use the same family for both jobs.

- Where does your system need public-key cryptography at all, and what would be lost without it?
- Which curve did you select, and what governed that choice: the security level it offers, the arithmetic it demands, the availability of a clear description, or resistance to implementation mistakes?
- Elliptic curve operations are cheaper than their finite-field counterparts at equal security. Quantify that advantage for your parameters rather than asserting it.
- Which parameter sizes, and what security level do they correspond to? Compare it against your symmetric choice — a 256-bit symmetric key behind a 1024-bit modulus is a design error, and we will ask about it.

7.4 Trusting a Public Key

This is the hardest problem in the project, and the one most often overlooked. If Layla asks the server for Omar's public key and the server substitutes its own, every later guarantee collapses.

Options: out-of-band fingerprint comparison between users; trust-on-first-use with change detection afterwards; a small certificate authority that you implement and that signs user keys; a short verification code derived from both parties' keys and compared by other means.

- Which approach did you choose, what does it assume of the users, and what happens the first time two people interact?

7.5 Signatures Versus MACs

- Where does your system use a message authentication code, where a digital signature, and why is each the appropriate tool at that point?
- Which of the two can satisfy the non-repudiation requirement in Section 6 — and why can the other not, however long its tag?

7.6 Architecture and Formats

Options: a client and a server communicating over sockets; a client and a server communicating through files in a shared directory that simulates the network; peer-to-peer with the server acting only as a directory. Any of these is acceptable; a simulated network is entirely adequate and will not cost you marks.

- What exactly does the server do, and what is it structurally prevented from doing?
- What is the byte-level format of your stored objects and your protocol messages? A precise format table must appear in your report.
- What does a client do when verification fails, and what does it report to the caller? Consider whether a detailed error message assists an attacker.

8. What “Done” Looks Like

A demonstration that satisfies every line below constitutes a complete project. Anything further counts as bonus work.

#	Your demonstration shows...
1	Two users register and log in on separate machines or separate directories.
2	The stored credential record, shown on screen, reveals nothing useful about either password.
3	A user uploads a document; the server's stored copy is displayed and is unreadable.
4	The document is shared with the second user, who downloads and opens it successfully.
5	The recipient's client states who produced the document, and the report explains how a third party could confirm it.
6	A single byte of the ciphertext is altered — the client rejects it cleanly.
7	A field of the metadata is altered — the client rejects it cleanly.
8	An old captured object is replayed — the client rejects it as stale.
9	Every primitive you implemented is shown to produce correct results, verified live.

Deliverables, all in your repository: the source code; a README with team members and instructions for running the system and the tests; the test suite; the design report of Section 10; and your presentation slides.

9. Implementation Rules — From Scratch

At least 70% of the cryptographic algorithms and protocols your system employs must be written by your team. You may not call a cryptographic library for those components.

The remaining 30% may come from libraries, provided your report names those components and says why you chose not to implement them. Big-integer arithmetic, randomness from the operating system,

networking, storage and interface code are not cryptographic components; take them from libraries freely and they do not count against your fraction.

Testing is mandatory. Every primitive you implement must come with tests that demonstrate it is correct, and those tests must be in your repository. An implementation you cannot show to be correct counts as not implemented. Cross-checking your output against a reference library inside your test files is encouraged and always permitted; how you establish correctness is itself a decision worth explaining in your report.

10. The Design Report

Six to ten pages, held in your repository, containing:

1. Threat model — who the attacker is, what they can do, what you have placed out of scope.
2. Design — architecture, byte-level message and file formats, key hierarchy, and the life of a document from upload to verified retrieval.
3. Justification — for every decision in Section 7: what you chose, what you considered, why you rejected it.
4. Security argument — for each property in Section 6, the mechanism that provides it.
5. Limitations — attacks your design does not stop, and what it would take to stop them. An honest limitations section earns marks; a claim of unconditional security loses them.
6. The AI usage statement of Section 14, and a contribution statement from each member.

10.1 What a Justification Looks Like

Below are two versions of the same decision. The difference between them is, in essence, the whole assessment.

Too weak: *“We hash passwords with a secure hash function, which protects them if the database is stolen.”*

Strong: *“We store credentials with a memory-hard password hash at a work factor tuned so one verification takes about 0.25 sec on the lab machines. Against an eight-character alphanumeric space, an attacker with one high-end GPU would need roughly N years per account under our parameters, versus about five minutes with a bare hash — the arithmetic is in Appendix B. We rejected the non-memory-hard alternative because GPU parallelism recovers most of the slowdown. We accept that a user who chooses a password from a common word list is still exposed within days; a dictionary check at sign-up would address this and is listed under limitations.”*

The strong version names the choice, gives a measured parameter, quantifies the benefit, names the rejected alternative with a reason, and admits what it does not solve. Aim for that in every decision.

11. A Suggested Way to Work

The schedule is short, so sequence matters. The plan below is advice rather than a requirement, but teams that begin coding before they have decided usually find themselves rewriting.

Stage	Work
Days 1–2	Decide. Work through Section 7 as a team and write your answers down before writing code. Sketch your formats on paper. This is the stage that determines your mark.
Days 3–6	Build and test your primitives one at a time. Do not proceed past a primitive you cannot show to be correct.
Days 7–9	Assemble the system: sign-up, login, key handling, upload, share, retrieve. Get the honest path working end to end, then break it deliberately and confirm it refuses.

Stage	Work
Days 10–11	Bonus work, the report, and the slides. Rehearse the demonstration, including the failure cases — those are the ones that go wrong live.

Commit as you go. The repository history is part of the evidence that this was your work.

12. Bonus Work

Bonus work is considered only when the core requirements are complete and the presentation demonstrates genuine understanding.

12.1 Graphical Interface

A usable graphical client instead of a command line. Marks are given for interface decisions that carry security weight — how a key fingerprint is presented so two users can actually compare it, how a verification failure is shown so a user cannot dismiss it by reflex, and which actions the interface refuses to permit at all.

12.2 Attacks: Implementation and Testing

Write an attacker as a separate, clearly marked program, and report what happens. Suggestions:

- A man-in-the-middle that substitutes public key material during a share, against your key-trust mechanism from Section 7.4.
- A tampering attack on ciphertext and on metadata, showing both are refused.
- A replay of a captured object.
- **Most instructive:** a deliberately weakened build of your own system — a repeated nonce, or the authentication step removed — with an attack that succeeds against the weak build and fails against the correct one. This demonstrates that you know why the defence is there.
- A measured estimate of how long an offline guessing attack on your credential store would take, with the arithmetic shown.

Attack code must target only your own system. Pointing any of it at university infrastructure or third-party services is a disciplinary matter.

13. Teams, Registration and Repository

Teams of three students. **Members may come from any section.**

Registration deadline: Monday, 7 September 2026. Register using the form below, giving the **full name and student ID of every member** and the link to your private GitHub repository.

Registration form: [Google Sheets registration link](#)

- Create the private repository as soon as your team forms.
- Invite the instructor as a collaborator at **ashawahna23@gmail.com** before the deadline.
- All code, the report, the slides and every other project file live in that repository.
- Commit and push regularly from each member's own account, with meaningful messages.
- **Do not commit private keys, passwords or other secrets.** Generate them locally and document the procedure in the README.

The commit history will be reviewed. One large commit shortly before the deadline indicates the work was not carried out as required; it will cost marks in the code component and may be raised as an

academic-integrity matter. Steady, incremental commits from all three members are what we expect to see.

14. Use of AI Tools

AI assistants are permitted under the below conditions. Misuse is treated as academic dishonesty.

Permitted:

- Debugging, reading error messages, looking up syntax.
- Explaining a concept that you then verify independently — as you would with a tutor.
- Feedback on the clarity and grammar of your report and README.
- Boilerplate and utility code: sockets, argument parsing, file handling, interface scaffolding, test harnesses.

Not permitted:

- Having an AI write your cryptographic primitives or your protocol logic.
- Having an AI make the decisions of Section 7 on your behalf. **Those choices must be your team's, reasoned by your team.**
- Submitting AI-generated text as your own report, analysis or conclusions.
- Presenting explanations you have not independently understood.

Mandatory AI usage statement. A dedicated section of the report must state which tools were used, exactly how, and which parts were produced entirely without AI assistance. Teams that used none must say so. Omitting this statement when tools were used is a violation of academic integrity.

You are answerable for every design decision, every claim and every line of code you submit. The presentation is built to test exactly that. Work you cannot explain is graded as though it were not yours.

15. Assessment

The project is worth 15% of the course grade.

15.1 Code — 5%

Criterion	Weight
Correctness of the implemented primitives, demonstrated by your tests	1.5%
Complete working system: sign-up, login, key management, upload, share, verify	1.5%
Compliance with the from-scratch rule, and honest declaration of library use	1.0%
Code quality, repository hygiene, commit history, README and report	1.0%

15.2 Presentation — 10%

Criterion	Weight
Design rationale: the choices of Section 7, the alternatives, and the reasoning	3.0%
Threat model and security analysis, including honest limitations	2.0%
Live demonstration, including the failure cases of Section 8	2.0%
Individual questioning: each member answers separately on any part of the work	2.5%
Clarity, structure and time management	0.5%

Each member is questioned individually. Team members normally receive the same mark; the instructor may differentiate where contribution is clearly imbalanced, or where a member cannot explain work attributed to them.

16. Presentations

Thursday 17 and Saturday 19 September 2026. A slot reservation link will follow. All three members must attend.

Be ready to run the system, to open any file in your repository, and to answer questions of this kind: why this construction and not that one; what stops a nonce repeating, and show me the code; which of your mechanisms gives non-repudiation and which cannot; alter one byte of the metadata now and show us what happens; how long would an offline attack on your credential store take, and how did you arrive at that number.

17. Academic Integrity

Full collaboration within a team is expected. Between teams, the following applies:

- You may consult standards, textbooks, pseudo-code and educational material, provided that every source is documented in your report. Consulting a reference is permitted; copying an implementation is not.
- You are graded on what your team understood, built and analyzed beyond what any source provided.

The following result in a grade of zero and possible disciplinary action:

- Plagiarism of text, code or results from any source, including unattributed AI-generated content.
- Sharing your materials with another team, or using another team's.
- Fabricating test results, or claiming vectors pass when they do not.
- Fabricating a commit history.

When in doubt, ask.

18. Key Dates

Milestone	Date
Team registration; repository created; instructor invited	7 September 2026
Design decisions of Section 7 written down and committed	12 September 2026
Primitives implemented and demonstrably correct	16 September 2026
Final commit; repository frozen	18 September 2026
Presentations	17 and 19 September 2026

Late submissions are not accepted without prior written approval from the instructor.

By the end of this project, you will have built the machinery that protects a stored file — and, more importantly, you will be able to say exactly why you built it that way rather than another. That second part is what distinguishes an engineer from someone who can run a library.

Good Luck